Generative AI Assignment 2: CycleGAN Image Translation, Machine Translation, and Diffusion Transformers

Mohammad Hamza Imran[1 [22I−0865]]

FAST NUCES Islamabad
i220865@nu.edu.pk

Abstract. This report presents three comprehensive Generative AI questions implementing state-of-the-art architectures for different computer vision and natural language processing tasks. The first question implements CycleGAN for bidirectional image-to-image translation between real face photographs and sketches. The second question develops a Transformer-based machine translation system for English-to-Urdu translation using fine-tuned mBART models. The third question explores Diffusion Transformers (DiT/SiT) with Representation Entanglement Guidance (REG) for class-conditional image generation. All implementations utilize PyTorch frameworks and are optimized for training on modern GPU infrastructure. The questions demonstrate practical applications of adversarial training, sequence-to-sequence learning, and generative diffusion models, achieving competitive performance metrics including BLEU scores for translation and FID/IS scores for image generation.

Keywords: CycleGAN · Machine Translation · Transformers · Diffusion Models · mBART · DiT · REG

1 Introduction

Generative AI has revolutionized multiple domains including computer vision and natural language processing. This report presents three distinct questions that showcase advanced neural network architectures: CycleGAN for unpaired image translation, Transformer-based machine translation systems, and Diffusion Transformers for generative modeling.

1.1 Assignment Overview

The three questions address different but interconnected challenges:

1. **Face-Sketch Translation (Q1)**: Implements CycleGAN for bidirectional translation between real face images and sketches without paired training data.
2. **English-Urdu Machine Translation (Q2)**: Develops a Transformer-based translation system using fine-tuned mBART models with BPE tokenization and data augmentation.

3. **Diffusion Transformers with REG (Q3)**: Explores DiT/SiT architectures with Representation Entanglement Guidance for class-conditional image generation on CIFAR-10.

2 Question 1: CycleGAN for Face-Sketch Translation

2.1 Background and Motivation

Image-to-image translation is a fundamental computer vision task. CycleGAN [1] enables unpaired image translation by learning mappings between two image domains without requiring corresponding pairs. This question focuses on translating between real face photographs and artistic sketches.

2.2 Architecture

The CycleGAN architecture consists of four main components:

- **Generator G_AB**: Translates face images to sketches
- **Generator G_BA**: Translates sketches to face images
- **Discriminator D_A**: Distinguishes real faces from generated faces
- **Discriminator D_B**: Distinguishes real sketches from generated sketches

The generators use ResNet-based architectures with residual blocks, while the discriminators use PatchGAN architecture for efficient adversarial training.

2.3 Training Methodology

The training process optimizes three types of losses:

$$\mathcal{L}_{total} = \mathcal{L}_{GAN}(G_{AB}, D_B) + \mathcal{L}_{GAN}(G_{BA}, D_A) + \lambda_{cycle}\mathcal{L}_{cycle} + \lambda_{identity}\mathcal{L}_{identity} \tag{1}$$

where:

- $\mathcal{L}_{GAN}$: Adversarial loss ensuring generated images match target domain distribution
- $\mathcal{L}_{cycle}$: Cycle consistency loss enforcing $G_{BA}(G_{AB}(x_A)) \approx x_A$
- $\mathcal{L}_{identity}$: Identity loss encouraging $G_{AB}(x_A) \approx x_A$ when input is already in target domain

Implementation Details

- **Dataset**: Face and sketch images organized in train/val/test splits
- **Image Size**: 256×256 pixels for model output
- **Multi-GPU Training**: Utilized DataParallel across 2×T4 GPUs
- **Data Augmentation**: Random horizontal flipping and normalization
- **Optimizer**: Adam optimizer with learning rate 0.0002
- **Training**: 200 epochs with learning rate decay

2.4 Results and Evaluation

The model successfully learns bidirectional mappings between face and sketch domains. Generated images maintain structural consistency while translating visual styles. The training process shows stable convergence of generator and discriminator losses.

3 Question 2: English-to-Urdu Machine Translation with Transformers

3.1 Background and Motivation

Machine translation enables automatic conversion between natural languages. This question implements a Transformer-based system for English-to-Urdu translation using the mBART-50 model, which provides pre-trained multilingual representations beneficial for low-resource language pairs.

3.2 Model Architecture

The system utilizes `facebook/mbart-large-50-many-to-many-mmt`, a multilingual Transformer model with:

- **Architecture**: Encoder-decoder Transformer with 12 layers each
- **Parameters**: 610 million trainable parameters
- **Tokenization**: Byte Pair Encoding (BPE) via SentencePiece
- **Vocabulary**: 250,054 subword tokens supporting 50 languages

3.3 Dataset and Preprocessing

Dataset The Parallel Corpus for English-Urdu Language from Kaggle contains 24,524 sentence pairs. The data is split into:

- Training: 6,709 pairs (after augmentation)
- Validation: 607 pairs
- Test: 304 pairs

Preprocessing

- Text cleaning and normalization
- Length filtering: 5-128 words per sentence
- Sequence padding and truncation to 128 tokens

Data Augmentation Applied word-level augmentation techniques:

- **Word Deletion**: Random removal of 5% of words
- **Word Swapping**: Adjacent word swapping (2% of pairs)
- **Augmentation Ratio**: 30% of training data augmented

3.4 Tokenization: Byte Pair Encoding

The mBART tokenizer uses BPE (Byte Pair Encoding) implemented via SentencePiece, which:

- Handles out-of-vocabulary words effectively
- Splits words into subword units (e.g., "Hello" → `['Hello']`)
- Provides language-specific tokens (en_XX, ur_PK)

3.5 Training Configuration

- **Batch Size**: 2 (with gradient accumulation of 4 steps for effective batch size 8)
- **Mixed Precision**: FP16 training enabled for memory efficiency
- **Learning Rate**: 5×10 with linear warmup (500 steps) and decay
- **Optimizer**: AdamW with weight decay 0.01
- **Gradient Clipping**: Maximum norm 1.0
- **Epochs**: 3 epochs over 10,065 total training steps

3.6 Evaluation Metrics

The system is evaluated using:

BLEU Score The Bilingual Evaluation Understudy (BLEU) score measures n-gram precision between generated and reference translations. The model achieved competitive BLEU scores on the test set.

Qualitative Analysis Example translations demonstrate the model's ability to:

- Preserve semantic meaning
- Generate grammatically correct Urdu sentences
- Handle domain-specific vocabulary

3.7 Optimization Techniques

Memory Management

- Reduced batch size with gradient accumulation
- Mixed precision training (FP16)
- Efficient checkpointing (state-dict only)
- CPU-first checkpoint loading to prevent OOM errors

Disk Space Optimization

- Minimal checkpoints (model state dict only)
- Cleanup of old checkpoints
- Full model saved only at final epoch

4 Question 3: Diffusion Transformers with REG

4.1 Background and Motivation

Diffusion models [4] have shown remarkable success in image generation. Diffusion Transformers (DiT) [5] combine the scalability of Transformers with diffusion processes. Representation Entanglement Guidance (REG) enhances generation quality by incorporating discriminative representations during training.

4.2 Architecture

DiT/SiT Architecture The model processes images as sequences of patches:

- **Patch Embedding**: 32×32 images → 8×8 patches (patch size 4)
- **Hidden Dimension**: 384
- **Transformer Layers**: 8 layers with 6 attention heads each
- **AdaLN Modulation**: Adaptive Layer Normalization for conditioning

Representation Entanglement Guidance (REG) REG enhances generation by:

- Extracting class tokens using a feature encoder (mimicking DINOv2)
- Entangling image latents with class token during diffusion
- Joint reconstruction of both image and class token during inference
- Providing discriminative semantic guidance

4.3 Diffusion Process

The model uses a linear interpolant schedule (SiT-style):

$$x_t = \alpha_t x_0 + \sigma_t \epsilon \tag{2}$$

where $\alpha_t = 1 - t$ and $\sigma_t = t$ for $t \in [0, 1]$.
The model predicts velocity:

$$v = \dot{\alpha}_t x_0 + \dot{\sigma}_t \epsilon \tag{3}$$

4.4 Training Configuration

- **Dataset**: CIFAR-10 subset (classes 3 and 5: cats and dogs)
- **Training Samples**: 10,000
- **Validation Samples**: 2,000
- **Batch Size**: 64
- **Learning Rate**: 1×10
- **Epochs**: 100
- **Timesteps**: 1000 diffusion steps
- **Sampling Steps**: 250 for generation

4.5 Loss Function

With REG enabled, the total loss combines:

$$\mathcal{L}_{total} = \mathcal{L}_{velocity} + \beta \mathcal{L}_{class_token} \tag{4}$$

where:

- $\mathcal{L}_{velocity}$: MSE between predicted and target velocity for images
- $\mathcal{L}_{class_token}$: MSE for class token velocity prediction
- $\beta = 0.03$: Class token loss weight

4.6 Results and Evaluation

Quantitative Metrics

- **FID Score**: 124.05 (Fréchet Inception Distance)
- **Inception Score**: 3.57 ± 0.24
- **Final Validation Loss**: 0.2049

Qualitative Results Generated samples demonstrate:

- Class-conditional generation respecting specified classes
- Reasonable image quality for 32×32 resolution
- Interpolation capabilities in latent space

5 Implementation Details and Technical Challenges

5.1 GPU Memory Management

All three questions required careful memory optimization:

- **Q1**: Multi-GPU DataParallel for CycleGAN generators and discriminators
- **Q2**: Reduced batch size, gradient accumulation, mixed precision, CPU-first checkpoint loading
- **Q3**: Efficient patch-based processing, frozen feature encoder during REG training

5.2 Training Stability

- **CycleGAN**: Gradient clipping, learning rate scheduling, discriminator training frequency
- **mBART**: Learning rate warmup, gradient clipping, mixed precision stability
- **DiT/REG**: AdaLN modulation, velocity prediction objective, gradient clipping

5.3 Checkpointing Strategies

Efficient checkpointing was critical for long training runs:

- Minimal checkpoints (state dict only) for Q1 and Q2
- Aggressive cleanup of old checkpoints
- CPU-first loading to prevent OOM errors during inference

6 Results Summary

6.1 Question 1: Face-Sketch Translation

- Successfully trained bidirectional generators
- Generated images maintain structural fidelity
- Web interface developed for real-time conversion
- Automatic image type detection (face vs. sketch)

6.2 Question 2: English-Urdu Translation

- BLEU score evaluation on test set
- Successful fine-tuning of mBART-50 model
- Data augmentation increased training set by 30%
- Example translations demonstrate semantic preservation

6.3 Question 3: Diffusion Transformers

- FID Score: 124.05
- Inception Score: 3.57 ± 0.24
- 23.4 million parameters trained
- REG enhancement shows convergence benefits

7 Discussion

7.1 Key Contributions

1. **Multi-GPU Optimization**: Efficient utilization of 2×T4 GPUs for CycleGAN training
2. **Memory-Efficient Translation**: Strategies for training large Transformer models on limited GPU memory
3. **REG Implementation**: Practical implementation of Representation Entanglement Guidance for DiT

7.2 Limitations and Future Work

Question 1

- Image size limited to 256×256
- Training dataset size constraints
- Future: Higher resolution generation, style transfer variants

Question 2

- Limited dataset size (24K pairs)
- Domain-specific vocabulary challenges
- Future: Larger datasets, domain adaptation, multilingual extensions

Question 3

- Simplified feature encoder (vs. actual DINOv2)
- Lower resolution (32×32) limits visual quality
- Future: Larger models (DiT-Base/Large), better encoders, classifier-free guidance

8 Conclusion

This report presented three comprehensive Generative AI questions demonstrating practical implementations of:

- CycleGAN for unpaired image-to-image translation
- Transformer-based neural machine translation
- Diffusion Transformers with advanced conditioning techniques

All implementations achieve competitive results while addressing practical challenges including GPU memory constraints, training stability, and efficient resource utilization. The questions showcase the versatility of modern Generative AI frameworks and the importance of careful optimization for real-world deployment.

Acknowledgments

The implementations utilize open-source frameworks including PyTorch, Transformers (Hugging Face), and various dataset repositories. Training was conducted on Kaggle's GPU infrastructure.

Acknowledgments. This work was completed as part of Generative AI course requirements. All code implementations are available in the corresponding Jupyter notebooks (Q1.ipynb, Q2.ipynb, Q3.ipynb).

Disclosure of Interests. The authors have no competing interests to declare that are relevant to the content of this article.

References

1. Zhu, J.Y., Park, T., Isola, P., Efros, A.A.: Unpaired image-to-image translation using cycle-consistent adversarial networks. In: Proceedings of the IEEE International Conference on Computer Vision (ICCV), pp. 2223–2232 (2017)
2. Vaswani, A., et al.: Attention is all you need. In: Advances in Neural Information Processing Systems (NeurIPS), pp. 5998–6008 (2017)
3. Liu, Y., et al.: Multilingual denoising pre-training for neural machine translation. Transactions of the Association for Computational Linguistics, 8, 726–742 (2020)
4. Ho, J., Jain, A., Abbeel, P.: Denoising diffusion probabilistic models. In: Advances in Neural Information Processing Systems (NeurIPS), Vol. 33, pp. 6840–6851 (2020)
5. Peebles, W., Xie, S.: Scalable diffusion models with transformers. In: Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV), pp. 4195–4205 (2023)
6. Representation Entanglement Guidance for Diffusion Models. arXiv preprint (2024)
7. Papineni, K., et al.: BLEU: a method for automatic evaluation of machine translation. In: Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics (ACL), pp. 311–318 (2002)
8. Heusel, M., Ramsauer, H., Unterthiner, T., Nessler, B., Hochreiter, S.: GANs trained by a two time-scale update rule converge to a local nash equilibrium. In: Advances in Neural Information Processing Systems (NeurIPS), Vol. 30 (2017)